Design Document

Professor	*University Course*
Nome Professore	Nome del corso

Author	*Personal Code*
Nome 1	Matricola 1
Nome 2	Matricola 2
Nome 3	Matricola 3

Settembre, 2025

Contents

1. Introduction

1.1 Purpose

Triplo is a cross-platform mobile application for smartphones and smartwatches, conceived as a digital logbook for trekking activities.
The application aims to enhance the experience of hiking in mountain environments by combining route information, environmental data, and recording of personal activity.
Users can explore predefined trekking routes and access useful information about the paths, including navigation support and contextual data such as weather conditions.
At the same time, the application allows hikers to document their experiences by creating logbook entries enriched with photos and personal diary notes.
The platform also enables users to discover and follow other hikers, and to browse selected entries from their trekking diaries, enhancing the sharing of experiences and inspiration for future hikes.

During a hike, users can optionally complete challenges, such as time-based goals designed to enrich the trekking experience.
The following sections describe the system architecture, the main functionalities of the application, and the technological choices adopted in its development.

1.2 Scope

Triplo supports users during trekking activities by providing guidance, contextual information, and tools to document their experiences in mountain environments.
The application focuses on the Madonna di Campiglio area and offers a set of predefined trekking routes with different levels of difficulty.
During a hike, users can access information related to the selected route, including navigation support and environmental conditions that may influence the trekking experience.
The application also encourages engagement through optional challenges that users may complete along the route. Each trekking activity can be recorded in a personal digital logbook, where users can document their experience by adding photos and diary notes. Users can also browse selected entries from the logbooks of other hikers they follow, allowing them to discover new routes and gain inspiration from shared experiences.
The application aims to support outdoor exploration while enabling users to record, revisit, and share meaningful moments from their trekking activities.

1.3 Definitions, Acronyms, Abbreviations

- GPS Generic term used in this document to indicate satellite-based positioning systems available on the device (such as GPS, Galileo, and others)

- LTE (Long Term Evolution) is a mobile communication standard. LTE is also referred to as "4G", the fourth generation of mobile communication technology. LTE consists of a number of enhancements to the "Universal Mobile Telecommunications System" (UMTS) – the third generation of mobile communications.

1.4 References

1.4.1 Main References

- Design and Implementation of Mobile Applications Lectures and Slides
  - Introduction https://webeep.polimi.it/mod/resource/view.php?id=399280
  - Flutter https://webeep.polimi.it/mod/resource/view.php?id=399287
  - Android https://webeep.polimi.it/mod/resource/view.php?id=411641
  - iOS https://webeep.polimi.it/mod/resource/view.php?id=417791
  - Lectures https://webeep.polimi.it/mod/url/view.php?id=376880

1.4.2 Other References

- Flutter Documentation https://docs.flutter.dev/
- Firebase Documentation https://firebase.google.com/docs/
- OpenWeather API Documentation
  https://openweathermap.org/api/one-call-3?collection=one_call_api_3.0
- Severe Weather Alerts API
  https://www.weatherbit.io/api/alerts
- Oracle Documentation: What Is Model-View-Controller (MVC)?
  https://www.oracle.com/technical-resources/articles/javase/application-design-with-mvc
  html
- Deutsche Telekom Documentation: What is LTE?
  https://www.telekom.com/en/company/details/the-nine-most-important-facts-about-lte-606

2. Architectural Design

2.1 Overview

2.1.1 High-level components

The Triplo system is designed according to a modular architecture in which the main application logic is implemented on the client side, while backend functionalities are delegated to managed services and external APIs, as illustrated in Figures 2.1 and 2.2.

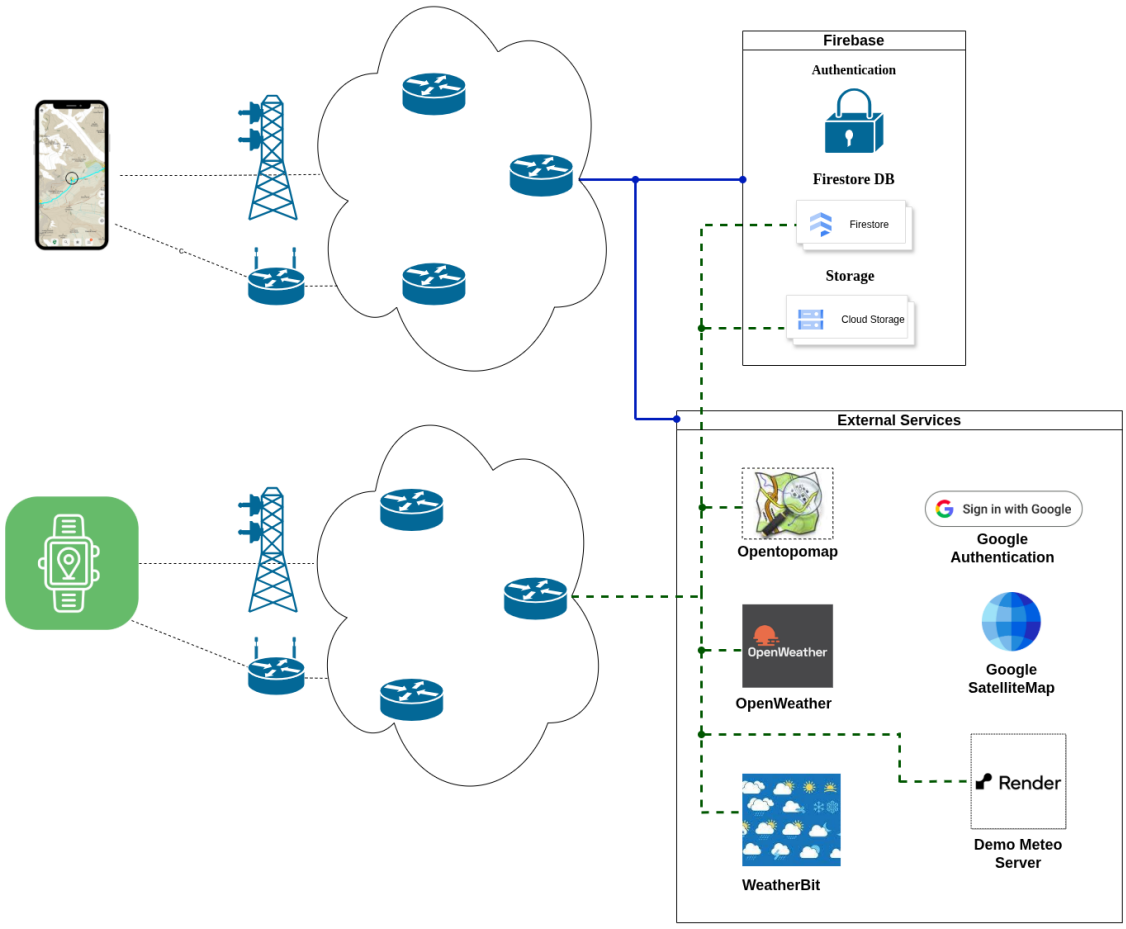


Figure 2.1: Overview Diagram: Blue lines denote the smartphone application's connectivity with Firebase and external services, providing full access to all Firebase modules and external APIs. Dashed green lines denote the smartwatch application's connectivity, which is limited to selected Firebase services, Firestore and Storage and specific external services, for example OpenTopoMap. The smartphone and smartwatch applications access the network independently, reflecting their ability to operate as standalone clients, while still relying on a shared set of backend services for data and functionality.

2.1.2 High level components interaction

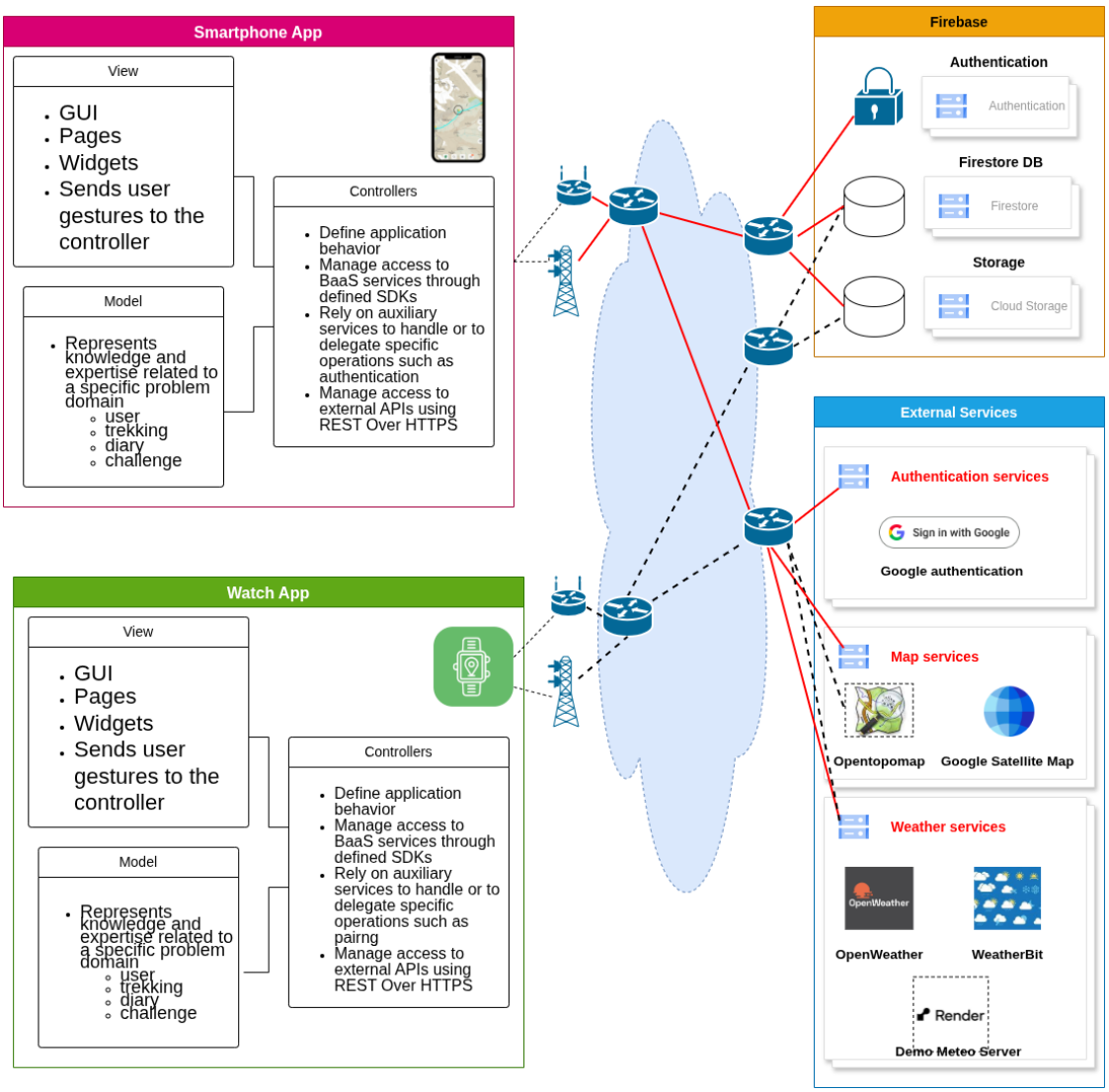


Figure 2.2: Detailed Overview Diagram

The system separates concerns between presentation, application logic, and data management by adopting a Model–View–Controller (MVC) architectural pattern. Both the presentation layer and the core application logic are implemented within the Flutter-based applications, while remaining logically decoupled through the use of Controllers that mediate between Views and Models.
In particular, the mobile application represents the main client, while the smartwatch application provides a lightweight extension offering a subset of functionalities adapted to the constraints and interaction model of wearable devices, relying on the same backend services and data sources.
User interactions are handled by the View layer, while business logic and coordination between components are delegated to Controllers, which act as the central mediation layer between the user interface and the underlying data models.
Data management and authentication are handled through Firebase services, specifically Firebase Authentication, Cloud Firestore, and Firebase Storage. The system leverages Firebase as a Backend-as-a-Service (BaaS) platform, providing managed services for authentication, data storage and user management, relying on platform-specific SDKs for access to these resources. The adoption of Backend-as-a-Service (BaaS) solutions enables the system to replace a traditional custom server-side layer with a more lightweight and scalable architecture. By delegating data management, authentication, and storage to managed services, architectural complexity is reduced by eliminating the need for maintaining a dedicated custom backend infrastructure. In addition to Firebase, the system integrates external third-party services (e.g., weather, map, and satellite APIs), which are accessed via RESTful APIs over

HTTPS.
This architectural approach emphasizes a client-driven design, reduces backend complexity, and enables scalability by leveraging managed cloud services.

This approach removes the need for a dedicated backend gateway or custom server-side components, allowing the application to directly orchestrate interactions with backend services and external resources.

2.2 Component view

2.2.1 High level component view

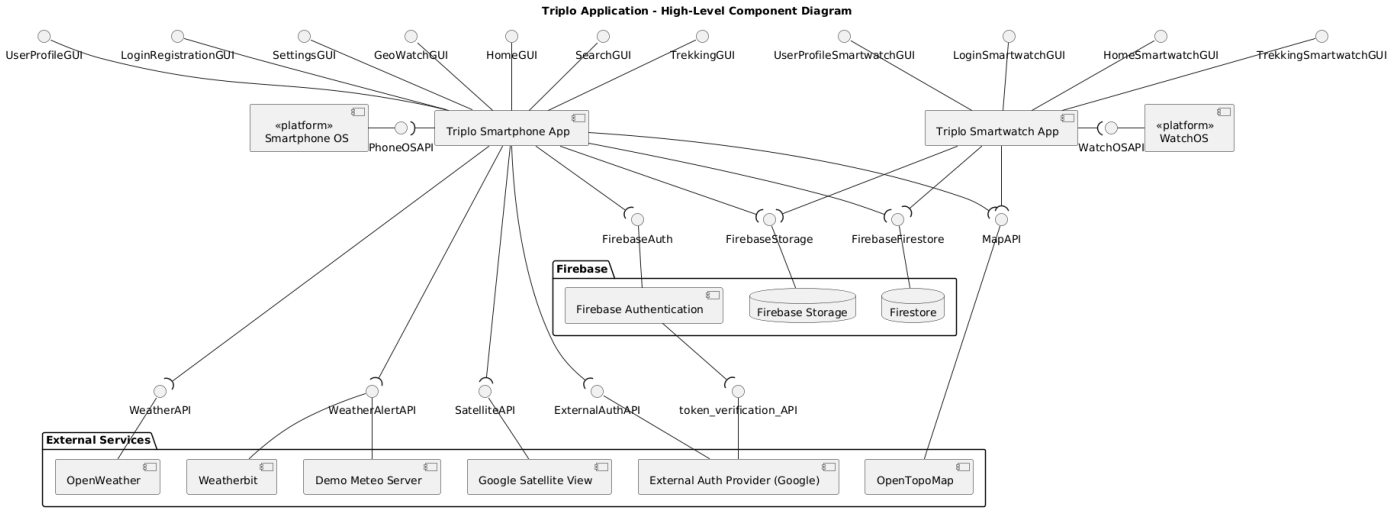


Figure 2.3: High level component view

The high-level component diagram provides an overview of the main architectural elements of the system and their interactions. The system is composed of two client applications, namely the smartphone application and the smartwatch application, both relying on a shared set of backend services and external resources.
The graphical user interfaces (GUIs) are represented following the convention adopted in both the *Software Engineering* and *UML Distilled* books quoted in the references. In this context, each GUI represents a group of related pages or a major functional area of the application, rather than a single page.
The diagram highlights the presence of two versions of the application. The smartphone application represents the main client and provides the full set of functionalities. It is implemented using Flutter, which enables cross-platform development and allows the application to run on both Android and iOS devices, as explained in section 2.7.
The smartwatch application represents a lightweight extension of the system and is deployed on Wear OS, as discussed in section 2.7 . It provides a subset of functionalities adapted to the constraints of wearable devices.
Both applications interact with a common set of backend services and selected external resources. In particular, both the smartphone and smartwatch applications access Firestore for data storage and Firebase Storage for media management, as well as shared external services such as OpenTopoMap for map visualization. However, the smartwatch application is intentionally restricted to a subset of services, reflecting its role as a lightweight client and the constraints of wearable devices.
An architectural difference regards authentication. While the smartphone application directly relies on Firebase Authentication, the smartwatch application does not perform authentication independently. Instead, it adopts a pairing mechanism based on QR code scanning. Through this process, the smartphone securely associates an authenticated user with the smartwatch by storing the corresponding

identifier in Firestore.
This design choice provides several advantages, detailed in section 2.7.3

Access to backend resources such as Firestore, Firebase Storage, and selected external services is otherwise shared between the smartphone and smartwatch applications. The diagram explicitly shows which services are accessible from each client, clarifying the distribution of responsibilities.
The system also integrates multiple external services, for example map providers such as OpenTopoMap and Google Satellite View, as well as external authentication providers. Weather alert functionalities rely on both Weatherbit and a Demo Meteo Server. The latter is used exclusively for demonstration purposes, allowing the simulation of weather alerts without depending on real external conditions.

The operating system interfaces are modeled as platform components, Smartphone OS and Watch OS. These platforms are not accessed directly by the application; instead, access is mediated through Flutter packages that provide abstractions over native operating system services. The specific packages used to interact with such services are detailed in the diagrams of the smartphone application components, section 2.2.2 and of the smartwatch application components, section 2.2.3.
The interfaces associated with Firebase services, Firebase Authentication, Firestore, and Firebase Storage are based on the SDKs provided by the Backend-as-a-Service platform. These SDKs abstract the underlying communication mechanisms and offer a structured way to access backend functionalities, as documented in the Firebase official documentation.
In contrast, the remaining external interfaces, such as Weather API, Weather Alert API, Satellite API, are accessed through RESTful communication. In these cases, the application uses HTTP-based requests via Flutter libraries, handling data exchange in JSON format and performing client-side decoding.
The token verification API is not directly managed within the application logic but is represented the complete authentication flow. In particular, when authentication is performed through an external provider, Google, the application obtains an authentication token which is then passed to Firebase Authentication. Firebase subsequently validates this token with the external provider as part of the OAuth2-based authentication process. This additional interaction is abstracted by the SDK but is included in the diagram to provide a more accurate representation of the overall system behavior.

2.2.2 Mobile Smartphone App component view

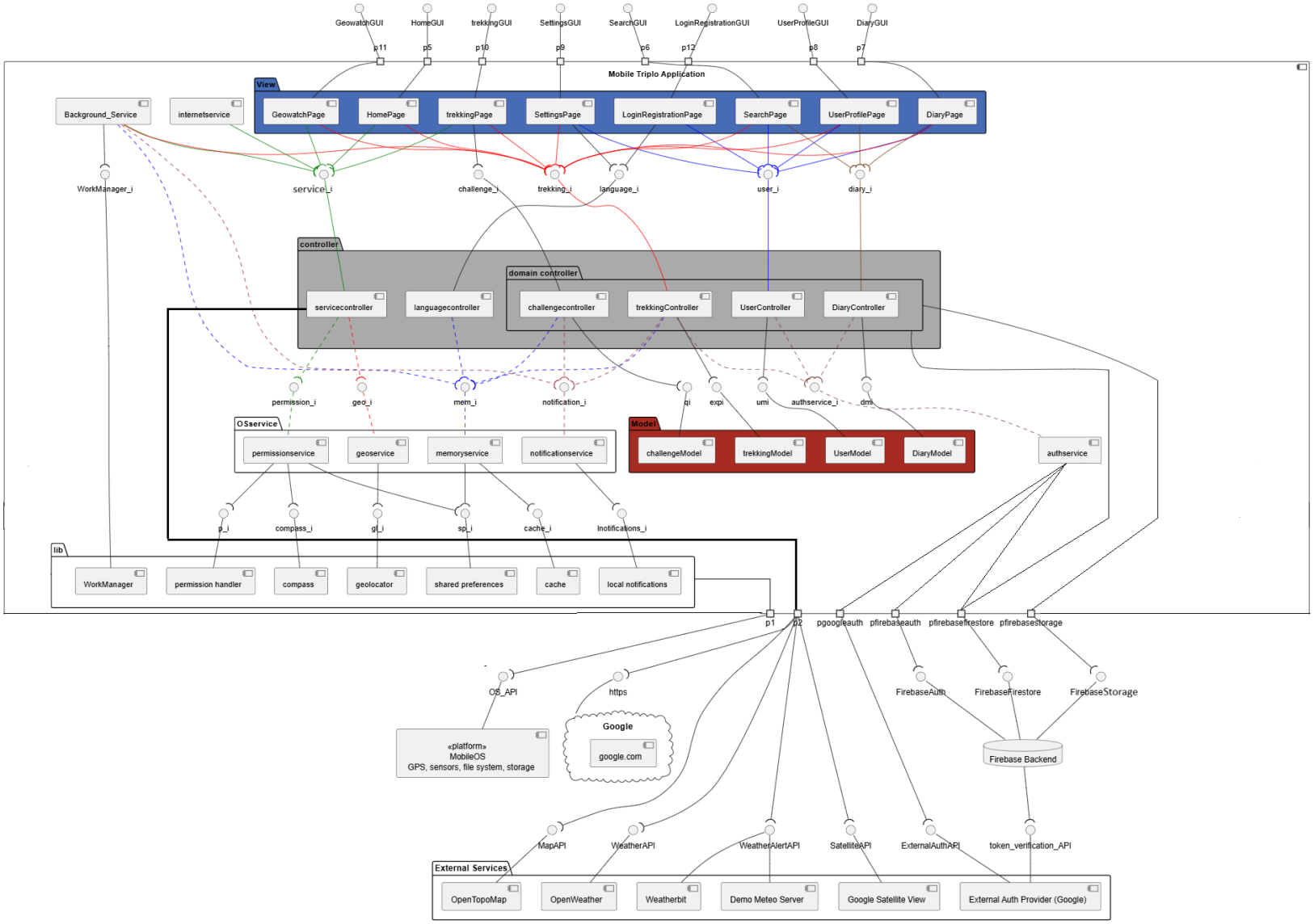


Figure 2.4: Smartphone Application component view

View Components

The UI of the application is organized into components that group together multiple views related to the same functionality, rather than representing isolated pages. This choice reflects how users interact with the system, as each View component encapsulates a complete feature.

The `GeoWatch Page component` includes all functionalities related to navigation and environmental awareness during a trekking activity. It is responsible for presenting weather information along the route, weather alerts, and a satellite map with meteorological layers. It also provides navigation support through compass heading, altitude, and user position.

The `Home Page component` represents the main map-based point of the application. It displays trekking routes on an OpenTopoMap layer and allows users to explore available paths geographically.

The `Trekking Page component` provides detailed information about a selected route, including a description and photos. It also allows users to access the component that handles the creation of a diary associated with the route and to save the route to their profile, the `Diary Page component`.

The `Diary Page component` handles the creation and modification of diary entries related to trekking experiences, including notes, images, and activity-related data.

The `User Profile Page component` is responsible for displaying user information, including saved trekkings, diary related, follower and following users' data.

The `Settings Page component` allows users to manage application and account settings, including profile information, security options such as password updates, smartwatch pairing, and weather notification preferences.

The `LoginRegistration Page component` manages login and registration workflows. It supports authentication via email and password, performs simple format validation operations, supports Google authentication, and password recovery.

The `Search Page component` enables users to search for other users and access their public profiles or to search for a trekking by its name.

Controller components

The controller package coordinates the application logic and mediates interactions between UI components, models, and backend services.

Domain controllers, such as `UserController`, `DiaryController`, `TrekkingController`, and `ChallengeController`, are responsible for handling use cases specific to their domain. They orchestrate data flow between the UI and Firebase services.

The `ServiceController` acts as a unified facade for functionalities that do not belong to an application domain defined in the model, such as access to external APIs and operating system services, reducing coupling between UI components and low-level implementations.

The `LanguageController` is responsible for managing localization and language-related logic.

OSService Package components

The OSService package contains services that directly interact with Flutter libraries responsible to communicate with system APIs and services managed by the operating system.

These include components such as `GeoService`, `PermissionService`, `MemoryService`, and `NotificationSe` each encapsulating a specific interaction with the operating system, such as location access, permission handling, local storage, and notifications.

This package isolates dependencies on external libraries and platform APIs. If a library needs to be replaced or updated, the changes are confined to the corresponding service, without affecting the rest of the application.

Internet Service

The `InternetService` is responsible for monitoring network connectivity. It periodically relies on the ServiceController to attempt connections to external endpoints (e.g., Google services). Based on the outcome, the application switches between offline and online modes, ensuring consistent behavior in varying network conditions.

Background Service

The `BackgroundService` enables the execution of tasks even when the application is not actively open, depending on operating system constraints.

It is used to perform periodic operations such as checking weather conditions and generating notifications. These tasks are scheduled through the mechanisms WorkManager mechanism, a Flutter wrapper around Android's Workmanager and iOS Background tasks. Background tasks are executed based on system availability.

AuthService

The AuthService is responsible not only for authentication but also for managing the smartwatch pairing process.

It handles user authentication through Firebase Auth and Google, password reset, session management, coordinates the pairing workflow between the smartphone application and the smartwatch, centralizing all identity-related logic.

Backend and External Services

The application relies on Firebase as a Backend-as-a-Service, including Authentication, Firestore, and Storage.

External services are used for specific functionalities, such as OpenTopoMap for map visualization, OpenWeather and Weatherbit for weather data and alerts, and Google services for authentication and connectivity verification.

Architectural Summary

The architecture is organized in packages that separate concerns clearly. Domain controllers manage application logic and backend interaction, the ServiceController provides a unified access point to external and system services, and OSService components encapsulate platform dependencies. This structure improves modularity, maintainability, and adaptability of the system.

Domain controllers, such as `UserController`, `DiaryController`, `TrekkingController`, and `ChallengeCon` are responsible for coordinating the application logic related to their respective functional domains. These controllers manage interactions between the View layer and the Model layer, and they also mediate access to backend resources such as Firebase services.

Therefore, in the proposed architecture:

- domain controllers manage the application use cases and the interaction with Firebase-related resources;
- The ServiceController acts as a unified facade aggregating multiple underlying services, including external APIs (weather, maps) and device-level services (location, permissions, navigation)
- Firebase is modeled as a set of backend services used by the application, rather than as a custom backend.

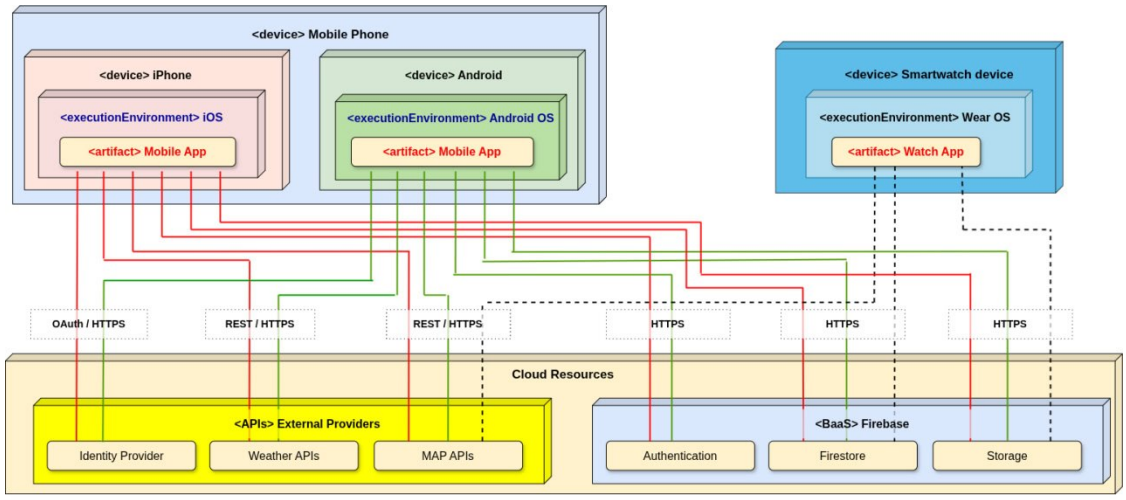
Smartphone Application External Libraries

Package	Purpose
flutter	Core UI toolkit for building cross-platform applications
provider	State management and implementation of the Observer pattern
firebase_core	Initialization of Firebase services
firebase_auth	User authentication management
cloud_firestore	Firebase cloud database for application data
firebase_storage	Cloud storage for images and media files
google_sign_in	Google authentication integration
flutter_map	Map rendering using OpenStreetMap tiles
latlong2	Geographic coordinate utilities
geolocator	Device location services
geocoding	Address and location conversion
flutter_compass	Compass and orientation data
image_picker	Image selection from device gallery or camera
mobile_scanner	QR code scanning functionality
shared_preferences	Local persistent key-value storage
flutter_local_notifications	Local notification management
workmanager	Background tasks
permission_handler	Runtime permission handling
flutter_dotenv	Environment variable management
uuid	Unique identifiers
intl	Internationalization and localization utilities
share_plus	Content sharing across apps
flutter_cache_manager	Image and file caching
cupertino_icons	iOS-style icon set

Table 2.1: Main dependencies used in the smartphone application

2.2.3 Mobile Smartwatch App component view

2.3 Deployment view



The deployment diagram is consistent with the UML deployment notation described in chapther 8 of the UML Distilled book and on page 52 of the Software Engineering book listed in the references. Physical hardware units are modeled through the `«device»` stereotype, while software platforms and runtime contexts are represented through the `«executionEnvironment»` stereotype.

The deployed software elements are shown as `«artifact»`. Communication paths are represented by links annotated with the corresponding protocol or interaction type, such as HTTPS, REST/HTTPS, and OAuth/HTTPS.

In particular, the diagram distinguishes the mobile deployment on Android and iOS devices, while the smartwatch deployment is limited to Wear OS. The cloud side is modeled as a device containing different execution environments, namely Firebase and the set of external providers, which host the artifacts used by the applications.

The deployment diagram represents the distribution of the application artifacts across different execution environments. In particular, the smartphone application is deployed as an artifact on both iOS and Android execution environments, while the smartwatch application is deployed separately on a Wear OS execution environment.

Although the smartphone application is shown as deployed on two distinct platforms, it is important to note that it is based on a single shared codebase. The application is developed using Flutter, a cross-platform framework that allows the same source code to be compiled and executed on both iOS and Android devices. As a result, the two deployments correspond to different execution environments of the same application, rather than distinct implementations. In contrast, the smartwatch application is implemented as a separate artifact. While it shares the same backend services and part of the architectural structure, it is specifically designed for the Wear OS environment and provides a reduced set of functionalities for wearable devices. Therefore, it is represented as a distinct deployment unit in the diagram.

2.4 Runtime view

2.4.1 Selected Use Cases

The identified use cases provide an abstract representation of the interactions between the user and the system. They have been selected to capture the essential functionalities of the application and to model its observable behavior from the user's perspective.
These set of use cases is representative of the main system capabilities and supports a clear understanding of the services offered to the user.

- Add Diary Page
- Explore trekkings on Map
- View trekking information
- View weather information
- Search Users
- Search trekking
- View compass and altitude information
- Participate in Route Challenge
- Login
- Register
- Pair Smartwatch
- Update User Profile

2.4.2 Add Diary Page use case sequence diagram

The most important part of the *Add Diary Page* sequence diagram is the save phase, where the data entered by the user is stored in the system.

When the user presses the save button, the `DiaryPage` starts the process by showing a loading page and then it formats the date, computing the duration from hours and minutes. If the user has selected photos, the page calls the `DiaryController` to upload them to `FirebaseStorage`. The controller uploads each image and returns a list of paths, which will be stored in the diary instance.

After this step, the `DiaryPage` calls the `addDiary(...)` method of the `DiaryController`, passing all the collected data. The controller retrieves the current user identifier from `FirebaseAuth`, generates a new unique identifier for the diary, and creates a `DiaryModel` object.

Before saving the diary in `FirebaseFirestore`, the object is converted into a `Map<String, dynamic>`. This is necessary because Firestore stores data in a JSON-like format, which is based on key-value pairs. The map represents the diary as a set of fields (such as date, duration, notes, etc.), where each field is associated with a value. The type `dynamic` is used because these values can have different types, such as strings, numbers, booleans, or lists.

Once converted, the map is stored in the `diary` collection in Firestore. At the same time, the controller updates the user document, adding the diary identifier to either the public or private diary list, depending on the selected visibility.

Finally, the control returns to the `DiaryPage`, which updates the user level through the `UserController` and navigates to the `UserPage`, completing the operation.

2.4.3 Watch–Phone Pairing Mechanism

Overview

The pairing mechanism associates a Wear OS device with an authenticated mobile user through a backend-mediated approval process. The watch does not authenticate directly with Firebase Auth; instead, device identity and user identity are linked via Firestore.

Device Identity

Each watch generates (or restores) a persistent `watchId`, stored securely on-device. A corresponding document exists in:

```
watch/{watchId}
```

The `watchId` uniquely identifies the physical device and is never deleted during logout.

Pairing Flow

1. Initialization (Watch) The watch generates a random UUID (`qrToken`) and writes:

```
watch_pair/{watchId}
```

with:

- `status = "waiting"`
- `uid = null`
- `qrToken`
- `createdAt`, `expiresAt`

A QR code encoding `watchId` and `qrToken` is displayed.

2. Approval (Phone) After scanning the QR, the authenticated mobile app:

- Verifies token validity and expiration.
- Updates the document with:

- `status = "approved"`
- `uid = request.auth.uid`

3. Completion (Watch) The watch listens to the document in real time. When `status == "approved"` and `uid` is set, pairing is considered complete and the watch loads user data using that `uid`.

Session Restoration

On startup, the watch checks:

`watch_pair/{watchId}`

If the document is already approved, the session is restored automatically.

2.4.4 Logout

Logout does not delete the device identity. Instead, the watch resets:

- `status = "waiting"`
- `uid = null`

This preserves device identity while removing user association.

2.4.5 Security Properties

- Only the watch may create/reset pairing in `waiting` state.
- Only authenticated users may approve pairing.
- QR tokens are random and time-limited.

This design cleanly separates device identity from user authentication while minimizing credential exposure on the wearable device.

2.5 Component interfaces

2.5.1 Component Interfaces Smartphone Application

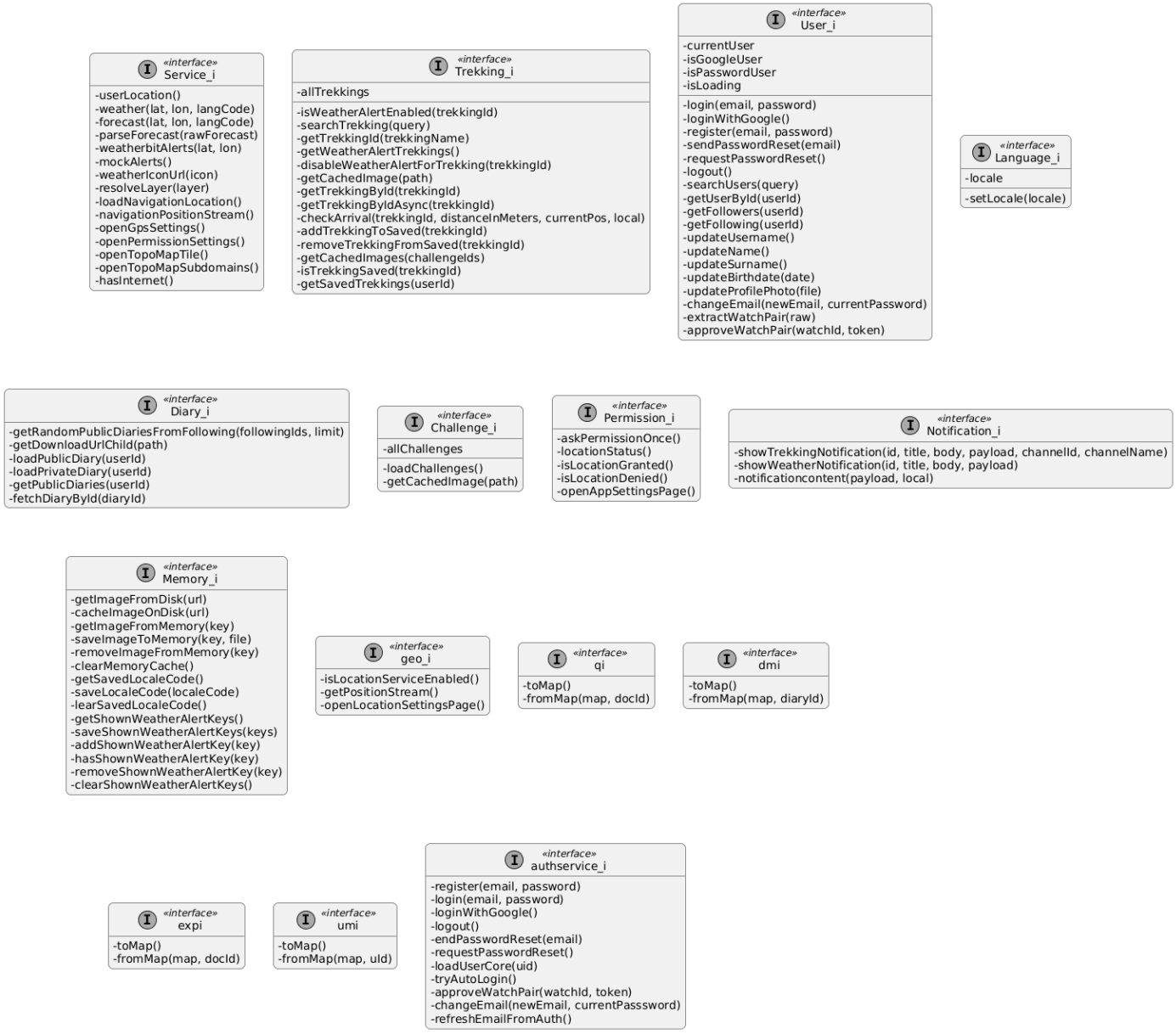


Figure 2.6: Interface diagram smartphone application

The interfaces shown in Figure 2.5.1 should not be interpreted as direct one-to-one representations of language-level constructs. Rather, they are used as an architectural abstraction to explicitly model the set of operations that each component exposes to the rest of the system. Figure 2.5.1 presents the main interfaces exposed by the application components. These interfaces define the contracts through which the different parts of the system interact, making the architecture more modular.

The diagram includes a first group of interfaces associated with the core application domains.

`User_i` defines the operations related to user profile management, following and followers data.
`Trekking_i` groups the operations required to retrieve trekking data, search routes, manage saved treks.
`Diary_i` exposes the operations related to diary retrieval and visualization.
`Challenge_i` provides access to challenge data.
Finally, `Language_i` defines the minimal contract required to manage the application locale.

A second group of interfaces regards infrastructure and platform interaction. `Service_i` acts as

the unified access point for a heterogeneous set of services that are not strictly tied to a single domain entity. As shown in the diagram, this interface includes operations for weather retrieval, forecast parsing, weather alerts, navigation-related location handling, access to map tiles, and redirection to system configuration components such as GPS and permission settings. For this reason, `Service_i` can be interpreted as a façade-oriented interface, collecting in a single access point the functionalities needed by the UI when interacting with external services and operating-system-level capabilities.

The operating system services are further represented through dedicated interfaces, namely `Permission_i`, `Notification_i`, `Memory_i`, and `geo_i`. These interfaces isolate the logic that depends on Flutter packages that communicate with device APIs or with the operating system. This design choice is particularly relevant from a maintenance point of view: if a package or platform-specific integration must be replaced, the impact can be limited to the corresponding service implementation, without requiring modifications in the rest of the application, as explained in section 2.7.4. In this sense, the interfaces shown in Figure 2.5.1 contribute to preserving a clear separation between application logic and platform-dependent concerns.

More specifically, `Permission_i` encapsulates permission requests.
`geo_i` abstracts location-service availability and position streaming
`Notification_i` defines the operations needed to issue local notifications
and `Memory_i` centralizes local storage concerns, including image caching, saving the preferred locale.

The diagram also contains a set of lightweight model-oriented interfaces, namely `qi`, `dmi`, `expi`, and `umi`. These define conversion contracts based on `toMap()` and `fromMap(...)` operations. Their purpose is to standardize the serialization and deserialization logic used to map application objects to and from Firestore representations.

In addition to the previously described interfaces, Figure 2.5.1 also includes the `authservice_i` interface, which encapsulates the operations related to user authentication and credential management.

This interface defines a dedicated contract for handling authentication workflows, including registration, login (both with email/password and Google Sign-In), logout, password reset, and automatic session restoration. Furthermore, it includes operations for account-related updates, such as email changes and smartwatch pairing approval.

From an architectural perspective, `authservice_i` plays a key role in separating authentication from the higher-level domain logic exposed by `User_i`. While `User_i` provides user-related functionalities at the application level, `authservice_i` is responsible for interacting with the underlying authentication provider (e.g., Firebase Authentication), thus centralizing all authentication-related operations in a single component.

This design choice improves modularity and maintainability, as it prevents other components from directly depending on authentication mechanisms and allows the authentication logic to be modified or extended with minimal impact on the rest of the system.

Overall, the interfaces represented in Figure 2.5.1 document a design in which each major responsibility is accessed through a clearly delimited contract. This improves readability of the architecture, supports separation of concerns, and makes explicit which services are expected from each component without exposing internal implementation details. For these reasons, the interface view complements the component decomposition by showing not only which elements exist in the system, but also how their responsibilities are formally made available to the rest of the application.

2.6 Selected architectural styles and patterns

2.6.1 Architectural Patterns: Model–View–Controller with Mediator and Observer

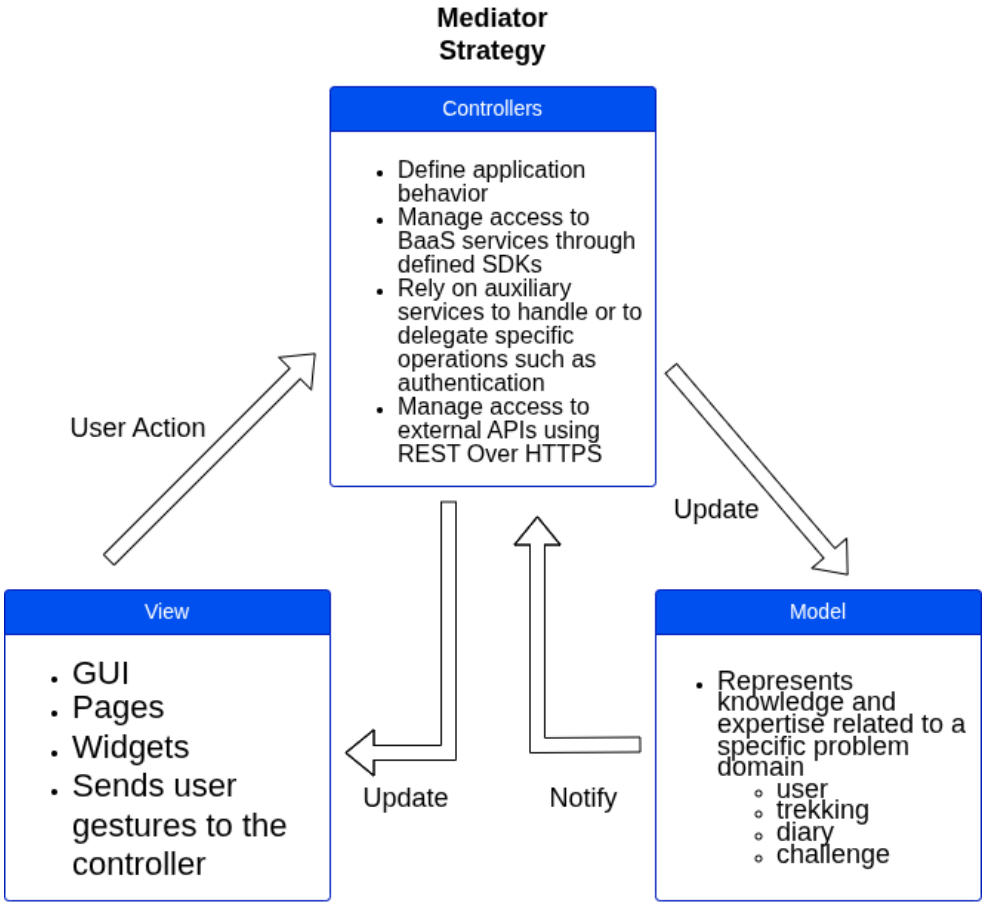


Figure 2.7: Model view controller pattern, adapted from Apple and Oracle documentation

The system adopts the Model–View–Controller (MVC) architectural pattern to separate presentation, application logic, and data management. Within this structure, each role is assigned a specific responsibility in order to improve modularity, maintainability, and clarity of interactions.

The **Model** represents the knowledge and expertise related to the application domain. It includes the core data structures of the system, such as `Diary`, `Trekking`, and `User`. The Model is independent from the user interface and does not handle user interactions directly.

The **View** is responsible for rendering the user interface and collecting user input. Its role is limited to presentation and interaction handling, without directly applying business logic or modifying domain data.

The **Controller** manages the application logic and coordinates the interaction between View and Model. In the adopted variant of MVC, consistent with the interpretation described in both Apple and Oracle documentation, the Controller also plays the role of a *Mediator*. More precisely, the View does not directly manipulate the Model; instead, user actions are forwarded to the Controller, which interprets them, executes the required logic, and updates the corresponding Model data. In this way, the Controller acts as the central coordination point of the interaction, reducing direct dependencies between presentation and domain data.

In addition, the architecture relies on the **Observer pattern** to keep the user interface consistent with state changes. When the state managed by a Controller is updated, the interested Views are notified through the listener mechanism provided by the Flutter framework, based on ChangeNotifier, integrated via Provider, so that they can refresh their representation accordingly. This notification-based interaction avoids tight coupling and supports consistency across multiple dependent Views.

Therefore, the adopted solution combines the structural organization of MVC with the use of the Mediator pattern in the Controller and the Observer pattern. This combination is particularly suitable for the Flutter-based implementation of the system, where the presentation layer, the application logic, and the domain data remain clearly separated while still cooperating through a well-defined interaction model.

2.6.2 ServiceController as a Facade

The ServiceController is designed following the Facade pattern.

It provides a unified interface that aggregates multiple underlying services, including external APIs and OS-level services. Instead of interacting directly with heterogeneous components, UI components and controllers rely on a single access point.

This design reduces coupling and simplifies the interaction model of the system. It also improves readability and consistency, as all non-domain-specific functionalities are accessed through a centralized component.

Furthermore, the facade allows internal changes to the underlying services without impacting the rest of the application, thus increasing flexibility and maintainability.

2.7 Other Design Decisions

2.7.1 Flutter as Application Framework

Flutter was selected for the implementation of the client applications because it provides an effective balance between cross-platform development, interface consistency, and maintainability. A primary requirement of the project was the possibility of deploying the smartphone application on both Android and iOS while preserving the same visual structure and interaction style across platforms. In this respect, Flutter offers a significant advantage, since it enables the development of a single shared codebase while also providing a uniform UI across platforms.

Compared with native alternatives such as Kotlin and Swift, Flutter avoids the need to maintain two separate implementations for Android and iOS, thus reducing duplication of development effort and simplifying maintenance. Compared with other cross-platform solutions, the choice of Flutter was particularly suitable for this project because it supports the construction of a coherent and controlled UI layer, which was important for preserving the same application identity and user experience on both mobile platforms.

2.7.2 Firebase as Backend-as-a-Service

The adoption of a Backend-as-a-Service solution represents a strategic architectural decision aimed at improving maintainability, scalability, and integration.
Firebase was adopted as the backend solution to support a scalable and robust system architecture while maintaining a clear separation between client-side logic and backend responsibilities. Rather than introducing a custom server-side layer, the system relies on a Backend-as-a-Service approach, where core functionalities are delegated to managed cloud services.

This choice allows the architecture to focus on the coordination of application logic on the client side, while relying on well-established and production-ready services for critical backend responsibilities such as authentication, data persistence, and media storage, integrating with the client applications through platform-specific SDKs.

From an architectural perspective, this approach improves modularity and reduces system complexity by eliminating the need to design and maintain a dedicated backend infrastructure. This approach presents notable advantages, including scalability, reliability, and security, through managed services that are designed to handle distributed workloads and concurrent access patterns.

Furthermore, the use of Firebase aligns well with the multi-platform nature of the system, as both smartphone and smartwatch applications can interact with the same backend services in a consistent and unified manner. This contributes to maintaining a coherent data model across different devices.

2.7.3 Phone Smartwatch Pairing

While the smartphone application directly relies on Firebase Authentication, the smartwatch application does not perform authentication independently. Instead, it adopts a pairing mechanism based on QR code scanning. Through this process, the smartphone securely associates an authenticated user with the smartwatch by storing the corresponding identifier in Firestore.
This design choice provides several advantages:

- avoids handling user credentials on the smartwatch or to enforce specific authentication methods, such as Google Sign-In, reducing the exposure of sensitive data on a constrained and potentially less secure device
- simplifies the user interaction model, eliminating the need for credential input on devices where text entry is inherently limited and less user-friendly

For these reasons, the interaction is intentionally defined as a pairing process rather than a traditional login, reflecting both its security properties and its role within the overall system architecture.

2.7.4 OSService Package Design

As shown in the smartphone component diagram (see Figure 2.2.2), the OSService package has been introduced to isolate all interactions with operating system functionalities and external libraries.

In particular, this package includes the following services:

- `PermissionService`, responsible for handling runtime permissions;
- `GeoService`, responsible for accessing device location;
- `MemoryService`, responsible for local storage and caching;
- `NotificationService`, responsible for managing local notifications;

Each service encapsulates the interaction with specific Flutter packages and platform APIs, such as geolocation, local storage, or notification systems. These dependencies correspond to external libraries (e.g., packages available on *pub.dev*) that internally communicate with the underlying operating system.

This design ensures that the rest of the application does not directly depend on platform-specific APIs or external libraries. Instead, all such dependencies are confined within the OSService package.

The main motivation behind this decision is maintainability. If a library or underlying implementation needs to be replaced, only the corresponding service must be modified, without requiring changes in the rest of the system. This significantly reduces the impact of technological changes.

2.7.5 Network Connectivity Requirements and smartwatch Platform

The smartwatch application requires reliable access to internet connectivity in order to provide a complete and consistent user experience. In particular, network access is necessary for several core functionalities of the system.

The application relies on external services to retrieve real-time data, such as maps, which are essential for trekking activities.

Internet access enables communication with backend services and APIs, allowing the application to fetch dynamic content and maintain updated information.

Given these requirements, the choice of Wear OS as the target platform is appropriate and justified. Wear OS supports multiple network communication modes, including:

- **Bluetooth connectivity**: allows the smartwatch to access the internet indirectly through a paired smartphone.
- **Wi-Fi connectivity**: enables the device to connect directly to wireless networks, allowing standalone operation in specific scenarios.

- **Cellular connectivity (LTE with eSIM)**: available on selected devices, providing full independence from the smartphone and ensuring continuous connectivity during outdoor activities.

This flexibility ensures that the application can operate under different conditions, ranging from smartphone-assisted usage to fully autonomous scenarios. In particular, the availability of LTE-enabled devices makes Wear OS especially suitable for outdoor and trekking contexts, where the user may not always have access to their smartphone.

3. Functional Requirements

3.0.1 Account and Profile Management

FR1 The system shall allow users to register by providing email and password

FR2 The system shall allow users to register using an external authentication provider

FR3 The system shall allow users to log in using email and password

FR4 The system shall allow users to log in using an external authentication provider

FR5 The system shall allow users to reset their password

FR6 When registration is successful, the system shall create a user profile

FR7 When registration is successful, the system shall assign a default username based on the user's email

FR8 The system shall allow users to update their username

FR9 The system shall allow users to update their profile image

FR10 The system shall allow users to update their name

FR11 The system shall allow users to update their surname

FR12 The system shall allow users to update their birth date

FR13 When users are registered with email and password, the system shall allow them to update their email address

FR14 When users are authenticated through an external provider, the system shall allow them to restore their profile image from the provider

FR15 The system shall allow users to change the application language

FR16 The system shall allow users to pair a smartwatch with their account

FR17 The system shall allow users to enable weather notifications for selected routes

3.0.2 Home, Routes and Route Details

FR18 The system shall display routes on a map

FR19 The system shall classify routes according to their difficulty level

FR20 The system shall display routes using different colors based on their difficulty level

FR21 When a user selects a route, the system shall display detailed information about the selected route

FR22 The system shall display route information including starting point, ending point, difficulty level, length, estimated time, and elevation gain

FR23 The system shall indicate the suitability of a route for families

FR24 The system shall display a textual description of the route

FR25 The system shall display refreshment points associated with the route

FR26 The system shall display challenges associated with the route

FR27 The system shall display summary weather information for the selected route

FR28 The system shall allow users to save a route to their profile

FR29 The system shall allow users to remove a saved route from their profile

FR30 The system shall allow users to start a navigation session for a selected route

FR31 When a navigation session ends, the system shall suggest creating a diary entry for the route

3.0.3 Diary Management

FR32 The system shall allow users to create a diary entry associated with a selected route

FR33 The system shall allow users to specify the date of the activity in a diary entry

FR34 The system shall allow users to specify the duration of the activity in a diary entry

FR35 The system shall allow users to select followed users who participated in the activity

FR36 The system shall allow users to add textual notes to a diary entry

FR37 The system shall allow users to indicate the presence of refreshment stops during the activity

FR38 The system shall allow users to select completed challenges in a diary entry

FR39 The system shall allow users to select a mood associated with the activity

FR40 The system shall allow users to upload images to a diary entry

FR41 The system shall allow users to mark a diary entry as public or private

FR42 The system shall allow users to save a diary entry

FR43 The system shall allow users to discard a diary entry before saving it

3.0.4 Search and Challenges

FR44 The system shall allow users to search for other users

FR45 The system shall allow users to search for routes

FR46 The system shall display search results according to the selected search type

FR47 The system shall display public profile information of searched users

FR48 The system shall display route information for searched routes

FR49 The system shall display diary entries from followed users

FR50 The system shall display the available challenges

FR51 The system shall display the requirements for completing each challenge

3.0.5 Profile and Social Features

FR52 The system shall display the user's profile picture, username, and level

FR53 The system shall display the number of diary entries created by the user

FR54 The system shall display the number of followers of the user

FR55 The system shall display the number of followed users

FR56 The system shall allow users to access the public profile of a follower

FR57 The system shall allow users to access the public profile of a followed user

FR58 The system shall allow users to share a public link to their profile

3.0.6 Navigation and Sensor Information

FR59 The system shall display compass information

FR60 The system shall display the current heading in degrees

FR61 The system shall display the user's current altitude

FR62 The system shall display the user's current geographic coordinates